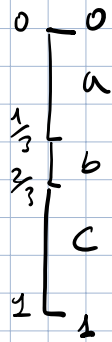


Dynamic Prefix Sums

Fenwick Tree (Binary Indexed tree, BIT)



a a a b
0 0 0 1

Problem

Given $A[1, n]$, we would like to support

- $Sum(i) = \sum_{k=1}^i A[k]$

- $Add(i, v) = A[i] += v$

($RangeSum(i, j) = Sum(j) - Sum(i-1)$)

Trivial Solutions

- Store A

- $\text{Sum}(i) = \sum_{k=1}^i A[k] \quad \Theta(n) \text{ time}$

- $\text{Add}(i, v) \rightarrow A[i] += v \quad O(1) \text{ time}$

- Precompute Prefix Sums P of A

- $\text{Sum}(i) \rightarrow P[i] \quad O(1) \text{ time}$

- $\text{Add}(i, v)$ increase any position $j \geq i$ in P by v $\Theta(n) \text{ time}$

BIT supports both in $\Theta(\log n) \text{ time}$

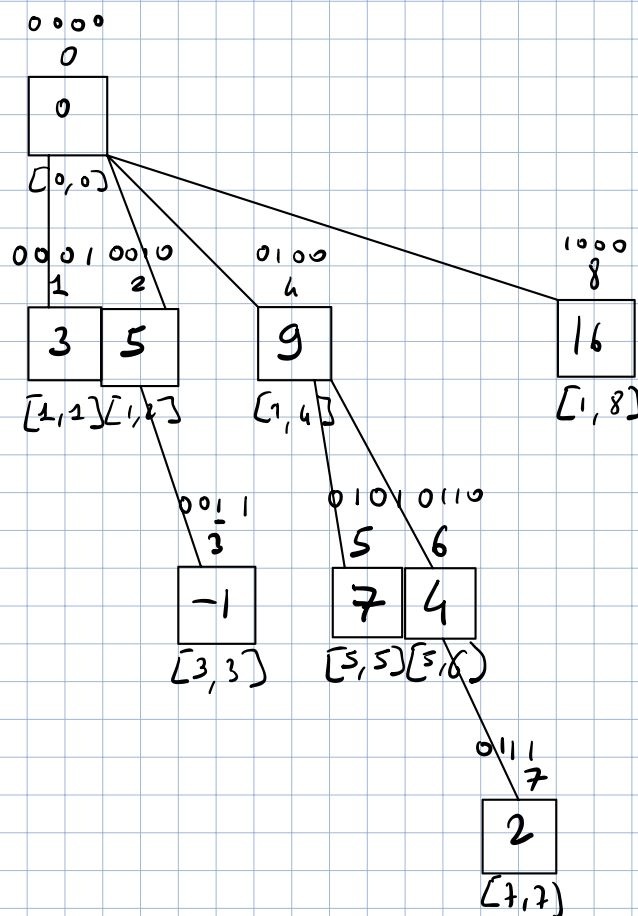
	1	2	3	4	5	6	7	8
A	3	2	-1	5	7	-3	2	1

Relaxed version of the problem :

- Summ(i) $\sum_{k=1}^i A[k]$ but only for positions i which are a power of 2

- Add(i, v) $A[i] += v$

	1	2	3	4	5	6	7	8
A	3	2	-1	5	7	-3	2	1



← positions with just one bit set to 1

← positions with Two bits set to 1

← positions with three bits set to 1

	1	2	3	4	5	6	7	8
FT	3	5	-1	9	7	4	2	16

$$\text{Sum}(7) = 2 + 4 + 9 = 15$$

		value
7	0 1 1 1	2
6	0 1 1 0	4
4	0 1 0 0	9
0	0 0 0 0	0

$$\text{parent}(i) = i - (i \& -i)$$

$$7 - (7 \& -7)$$

$$0111 - \left(\begin{array}{c} 0111 \& 1001 \\ \hline 0001 \end{array} \right) = 0110 = 6$$

$$i + (-i) = 2^N$$

$$\begin{array}{r} i \quad b_1 b_2 b_3 b_4 \dots b_n \boxed{1} 0 0 0 0 0 \\ -i \quad \bar{b}_1 \bar{b}_2 \bar{b}_3 \bar{b}_4 \dots \bar{b}_n \boxed{1} 0 0 0 0 0 \\ \hline \end{array}$$

Sum(i)

$$\Delta = 0$$

while $i \neq 0$

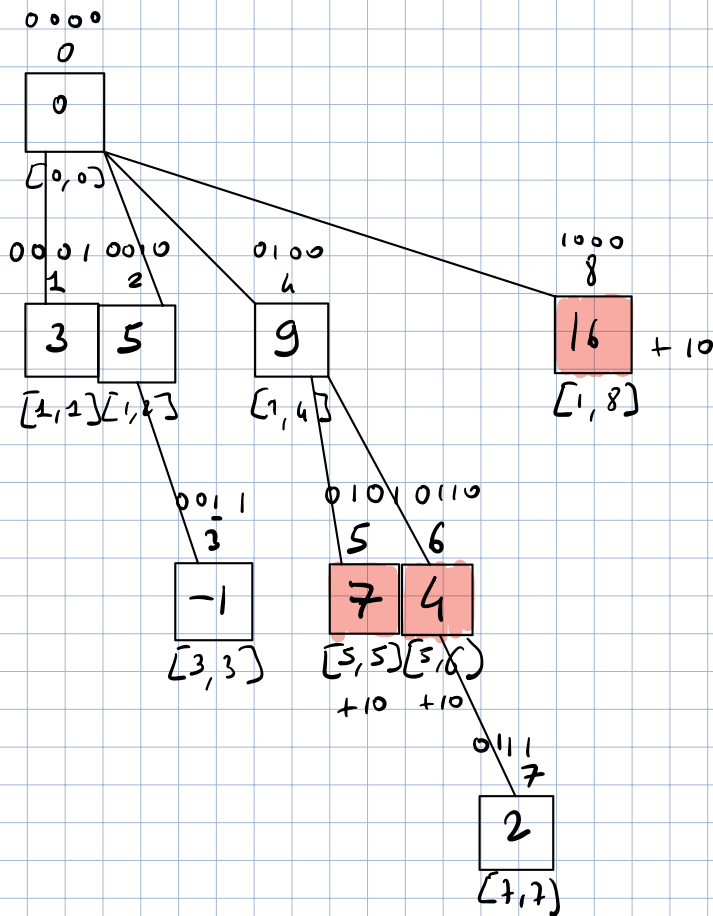
$$\Delta += \text{FT}[i]$$

$$i = \text{parent}(i) \quad // \quad i -= i \& -i$$

return Δ

Complexity : $\Theta(\log n)$ time

Add(5, 10)



5 0 1 0 1 + 0 0 0 1
 6 0 1 1 0 + 0 0 1 0
 8 1 0 0 0 + 1 0 0 0
 16 1 0 0 0 0

move_right(i) = i + (i & -i)

parent(i) = i - (i & -i)

Add(i, v)

while i ≤ n

A[i] += v

i += i & -i

Complexity :

Counting Inversions

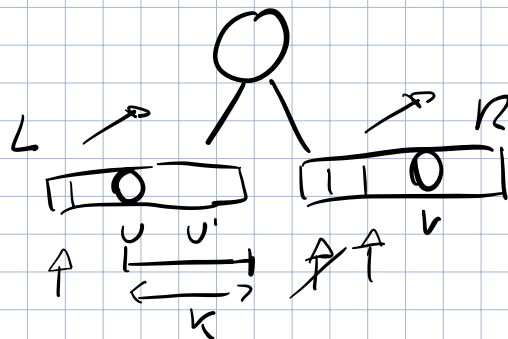
Given $A[1, n]$ of positive numbers $\leq n$,
Count the number of inversions in A

$i < j$ such that $A[i] > A[j]$

A $\underbrace{3 \ 2 \ 1}_{\text{3 inversions}} \quad \underbrace{6 \ 5 \ 4}_{\text{2 inversions}} \quad \# \text{ inversions} = 6$

Solution

- Running steps Insertion Sort can count number of inversions but $\Theta(n^2)$ time
- Merge Sort - Based solution in $\Theta(n \log n)$ time



if $v < u$, # - Inversions = k

- Solution based on Fenwick Tree

	1	2	3	4	5	6
A	3	2	1	6	5	4

→

F	0 1	1	0	0	0	
	1	2	3	4	5	6

$i < 3$ such that $A[i] > A[5]$

$$\text{So Range Sum}(1+1, 6) = 2$$

$$= 5 - \text{Sum}(5)$$

- $\Theta(n \log n)$ time

if max value is too big, remap

1	2	3	4	5	6
1	50	1,000,000,000	155	6	6

REMAPPING

- Sort in increasing order $\Theta(n \log n)$ time
- remap a value with its rank in

the ordering

pos rank

1 (, 1) 1

6 (, 5) 2

50 (, 2) 4

133 (, 4) 5

1B (, 3) 6

	1	2	3	4	5	6
A	1	50	1.000.000. 000	133	6	6

A	1	4	6	5	2	3
---	---	---	---	---	---	---

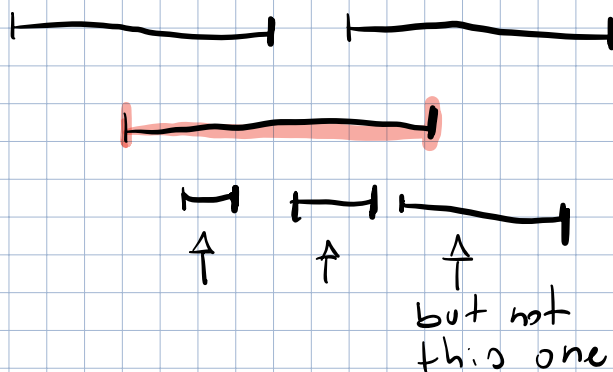
remap

Nested Segments

We are given segments

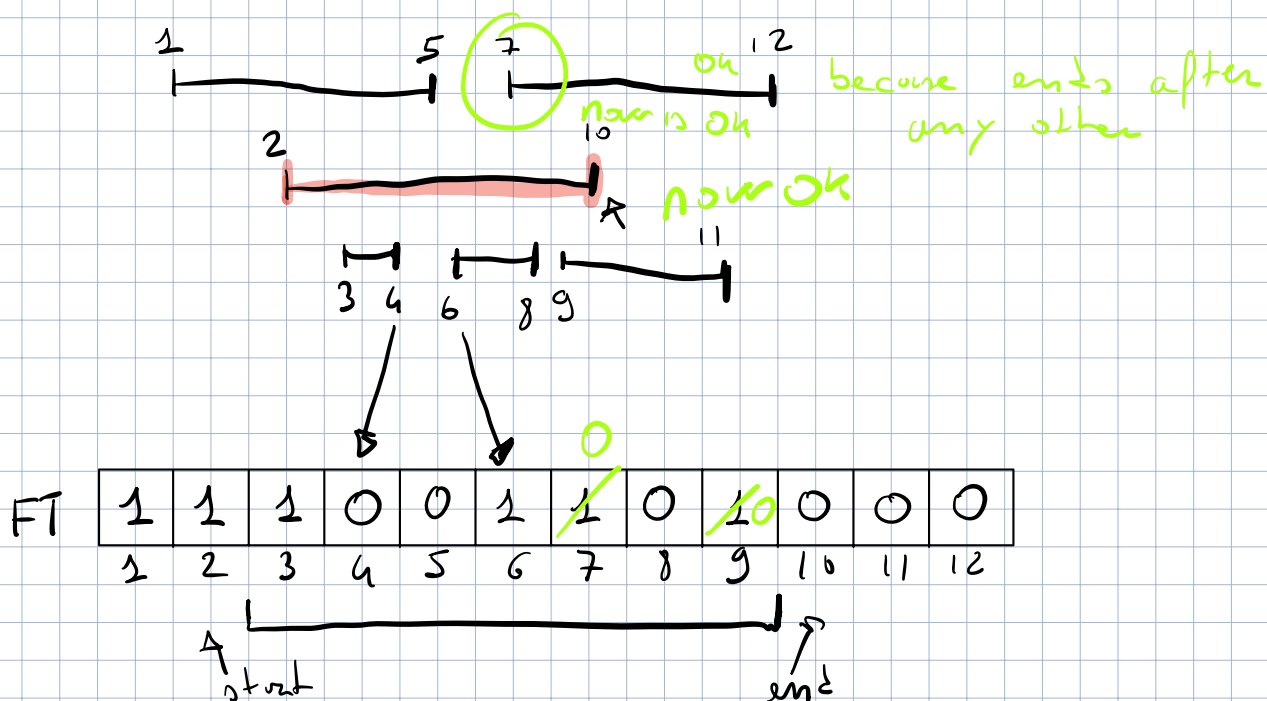
$$\langle l_1, r_1 \rangle, \langle l_2, r_2 \rangle, \dots, \langle l_n, r_n \rangle$$

The goal is to report, for each segment, the number of segments it contains.



Solution in $\Theta(n \log n)$ time

- Remap the extremes if needed



RangeSum(3, 9) = 4 = number of segments
starting within this

RangeSum(3, 9) = 2 segments

Update the array

Given $A[?, n]$ initially $A[i] = 0 \forall i$

we would like to support:

- $\text{Access}(i) = A[i]$

- $\text{Update}(i, j, v) \quad \forall k \in [i, j) \quad A[k] += v$

Example

	0	1	2	3	4	5	6	7	8
A	0	0	0	0	0	0	0	0	0
	5	5	10	10	10	10	10		
			15	15	15				

Update(2, 6, 10)

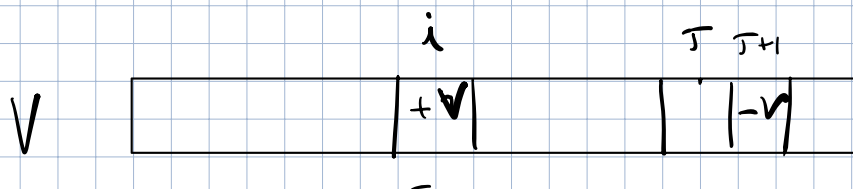
Update(0, 4, 5)

Solution: Both ops in $\Theta(\log n)$ time

- Trivial keep A and Update(i, j, v) in $\Theta(j-i)$ time

Idea: use $V[1, n]$ such that

$$\forall i: A[i] = \sum_{k=1}^i V[k]$$



Update(i, j, v)

	+5		+10			-5		-10	
V	0	0	0	0	0	0	0	0	0
	0	1	2	3	4	5	6	7	8
A	0	0	0	0	0	0	0	0	0
	5	5	10	10	10	10	10		
			15	15	15				

Update(2, 6, 10) ←

Update(0, 4, 5) ← $\begin{cases} \text{Add}_v(0, 5) \\ \text{Add}_v(5, -5) \end{cases}$

$$\text{Access}_A(i) = \text{Sum}_v(i)$$

both $\Theta(\log n)$ time with FT

Dynamic Prefix Sums with Range Update

Given $A[1, n]$

- Range Update (i, j, v)

- Sum (i)

[- Access $(i) = \text{Sum}(i) - \text{Sum}(i-1)$]

	1	2	3	4	5	6	7	8	9	10
A	0	0	0	0	0	0	0	0	0	0
				+10	+10	+10				

Range Update $(4, 6, 10)$

Sum $(6) = 30$

Solution runs $\Theta(\log n)$ time per query

Two Fenwick trees

FT₁ used in a similar way (as in the previous problem)

	1	2	3	4	5	6	7	8	9	10
V	0	0	0	0	0	0	0	0	0	0
				+10			-10			

Range Update $(4, 6, 10)$

$$\text{Sum}(i) = \text{FT}_1 \cdot \text{Sum}(i) \cdot i$$

Range Update (^l1, 6, 10)

Oh if $l == 1$

1 2 3 4 5 6 7
+10 -10

$$\text{Sum}(6) = \text{FT}_1 \cdot \text{Sum}(6) \cdot 6 = 60$$

Range Update ($l, r, +v$)

- $+v$ at position l

- $-v$ at position r

$$\text{Sum}(i) = \text{FT}_1 \cdot \text{Sum}(i) \cdot i$$

• What's wrong? correct

① if $0 \leq i < l$

0

$0 \cdot i = 0$

② if $l \leq i \leq r$

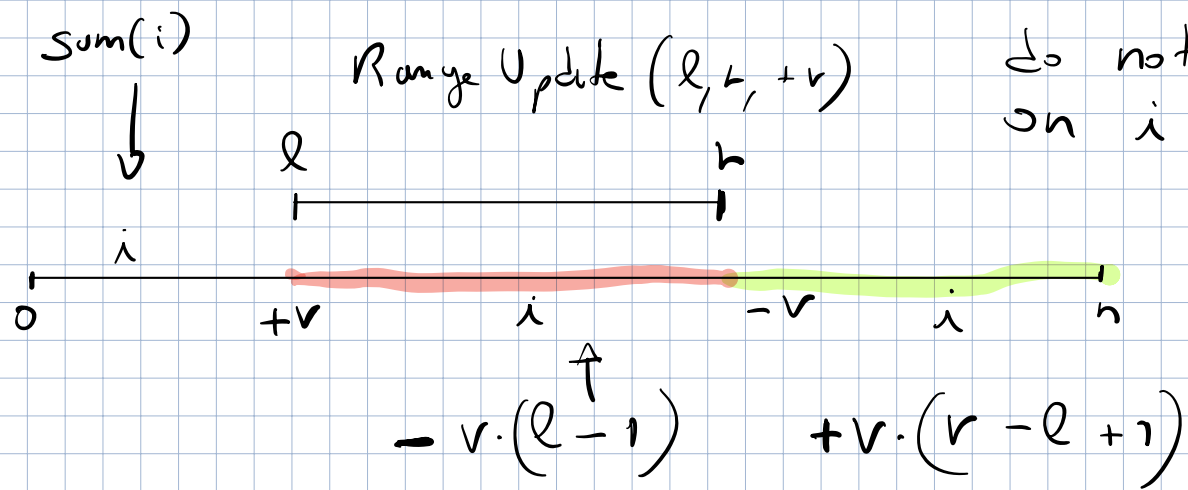
$v \cdot (i - l + 1)$

$i \cdot v = v \cdot (l - 1) + v(i - l + 1)$

③ if $r < i \leq n$

$v \cdot (r - l + 1)$

$0 \cdot i = -v \cdot (r - l + 1)$



both errors
do not depend
on i

Keep an array FT_2 of corrections with
Update the array solution

- $\text{FT}_2 \cdot \text{RangeUpdate}(l, r, -v(l-1))$
- $\text{FT}_2 \cdot \text{RangeUpdate}(r+1, n, v \cdot (r-l+1))$

$$\text{Sum}(i) = \text{FT}_1 \cdot \text{Sum}(i) \cdot i + \text{FT}_2 \cdot \text{Sum}(i)$$